

Queue Model Implementation - Practical Session

You will build a functional queue model based on Kendall's notation, which is a standard framework for describing and analyzing queueing systems.

Kendall's Notation Overview

Kendall's notation uses the format $A/S/c/N/D$ to fully characterize a queue:

Parameter	Meaning	Examples
A	Distribution of arrival times	M (Markovian/Exponential), D (Deterministic), G (General)
S	Distribution of service times	M (Exponential), D (Deterministic), G (General)
c	Number of servers	1, 2, 5, ... (integer values)
N	System capacity (optional)	Maximum entities in system; omit if infinite
D	Queue discipline (optional)	FIFO (First-In-First-Out), LIFO (Last-In-First-Out), SIRO (Service-In-Random-Order)

Example: $M/M/1$ = Exponential arrivals / Exponential service / 1 server (infinite capacity, FIFO assumed)

Your Implementation Task

Core Requirements

You must implement a **modular queue system** with the following characteristics:

1. **Flexible parametrization:** Your model must accept any valid Kendall notation parameters (A, S, c, N, D(optional))
2. **Modularity:** The system must be designed so that **different queue types can be connected together**. This is crucial, you'll expand to multi-stage systems in the next session.
3. **Event-driven architecture:** Use **Event Scheduling** as your simulation methodology
 - Events: Customer arrivals, service completions, queue state changes
 - Time advances are event-driven, not fixed-step
4. **Validation mechanism:** Implement a **GPSS-compatible model** that can validate your results against theoretical predictions or alternative simulation approaches.
5. **Validate the TRACE.**

The document

Executive summary

A brief and concise description of what is going to be analyzed, the problem and the solution proposed.

System description, introduction

This section only contains the description of the system to be modeled (not the problem, not the data), other elements must be described on the subsequent sections of the document.

Describe the system to model, the details that are relevant for the analysis.

Job scheduling and batch queue systems

- Description: Jobs (tasks or MPI jobs) arrive to an HPC cluster, wait in batch queues, acquire nodes/cores, run, then release resources.
- BPMN mapping:
 - o entry events = job arrivals (A by class).
 - o lanes = queue classes or partitions.
 - o service tasks = resource allocation + execution (B depends on job length and allocated cores).
 - o routing = retry/hold, move between queues (e.g., priority promotion), or exit on completion.
 - o capacity c = number of jobs the scheduler/partition can hold or number of available compute slots.
- Simulation focus: queueing delays, utilization, throughput, blocking when resources scarce.

Problem description

Describe the problem you want to solve using simulation. Be as clear and concise as possible. Be aware, the problem you discover through modeling can be different to the proposed problem by the client of the stakeholders. In brief, the problem is the problem that you detect through the model use.

What are the issues you detect?

Systemic Structural, Systemic Data and Simplifying Hypotheses

Complete the problem description with the hypotheses. Usually, they are going to be presented with an identifier.

SH_01 My simplification hypotheses 01

SH_02 My simplification hypotheses 02

SS_01 My structural hypotheses 01

SD_01 My data hypotheses 01

...

Model specification

Use BPMN to detail the different model components, represent each queueing node as a lane with its parameters {A, S, C, N (optional), D (optional)}, model external arrivals as Start Events annotated with arrival rates, model service as Tasks, and use Gateways with annotated probabilities for routing; annotate sequence flows with transit delays or mark them 'instant' and add data artifacts or notes for capacity/blocking rules. Attach a one-page legend mapping BPMN elements to simulation parameters for implementation.

Recommended BPMN tools (each supports PNG/SVG export and BPMN XML):

- Camunda Modeler — <https://camunda.com/products/camunda-platform/modeler/>
- bpmn.io (online editor) — <https://bpmn.io/>
- Bizagi Modeler — <https://www.bizagi.com/en/platform/modeler>
- Signavio Process Manager — <https://www.signavio.com/products/process-manager/>
- Visual Paradigm — <https://www.visual-paradigm.com/features/bpmn/>

Coding

Implement the model using C or Python. If you plant to use any other tool, please consult us.

Make an implementation of the model in GPSS for validation purposes.

Data

Describe the connection mechanism of the data used along the model.

Definition of the experimental framework

Explain the DOE to be used. Detail the process to execute the replications.

1. Define a DOE to explore with what parametrization of the factors the answer obtains the best value (define what means best, i.e. maximize or minimize the value).
2. Detect and analyze the interactions.

Model validation

Propose some methods to perform a validation of your model.

- Verify the main simulation algorithm implementation.
- Verify the RNG implementation and validate its parametrization.
- Validate the experimental framework.
- Validate the model assumptions.
- Validate the results.

Results /Conclusions

Be as concise as possible, describe the main conclusions of the analysis done.

Annexes

GNA/GVA

Generate RNG.

Test it using a well know battery, at least 5 tests.

Example using R with “randtoolbox” package.

title: "PRNG Validation with randtoolbox"

author: "Randomness Analysis"

output: html_notebook

Assessing Generator Suitability

This document implements a Linear Congruential Generator (LCG) and validates its statistical behavior through uniformity and independence tests using the `randtoolbox` package.

```
``{r}
```

1. Environment Setup and Library Loading

```
if (!require("randtoolbox")) install.packages("randtoolbox")
```

```
library(randtoolbox)
```

2. Linear Congruential Generator (LCG) Implementation

```
# We define the parameters for the algorithm:  $X_{n+1} = (aX_n + c) \bmod m$ 
```

```
m <- 2^31
```

```
a <- 1103515245
```

```
c <- 12345
```

```
seed <- 42
```

```
generate_lcg <- function(n, seed, m, a, c) {
```

```
  rng_values <- numeric(n)
```

```

current <- seed
for (i in 1:n) {
  current <- (a * current + c) %% m
  rng_values[i] <- current / m
}
return(rng_values)
}

```

Generate a sequence of 10,000 numbers

```
numbers_custom <- generate_lcg(10000, seed, m, a, c)
```

3. Helper Function for Automatic Interpretation

This function evaluates the p-value against a 0.05 significance level.

```

interpret <- function(result, test_name) {
  p_val <- result$p.value
  cat("\n===== \n")
  cat(" TEST:", test_name, "\n")
  cat("===== \n")
  print(result)

  if (p_val < 0.05) {
    cat("\nVERDICT: THE SEQUENCE DOES NOT APPEAR RANDOM.")
    cat("\nReasoning: The p-value (", p_val, ") is less than 0.05, indicating significant
patterns.\n")
  } else {
    cat("\nVERDICT: THE SEQUENCE APPEARS RANDOM.")
    cat("\nReasoning: The p-value (", p_val, ") is greater than 0.05; no evidence of
non-randomness found.\n")
  }
}
}

```

4. Execution of the Statistical Test Battery

A. Gap Test:

This test examines the length of "gaps" between numbers falling into a specific

range [l, u]. If the generator is random, these lengths should follow a

geometric distribution. It helps detect clusters or voids in the output[cite: 2].

interpret(gap.test(numbers_custom), "Gap Test")

B. Frequency Test:

Also known as the Monobit test, it checks the global uniformity of the sequence.

It ensures that numbers are distributed evenly across the [0, 1] interval

without favoring specific sub-regions[cite: 2].

interpret(freq.test(numbers_custom), "Frequency Test")

C. Order Test:

This test analyzes the relative order (permutations) of groups of 'd'

consecutive numbers. It is used to detect correlations in higher dimensions

that simple uniformity tests might miss[cite: 2].

interpret(order.test(numbers_custom, d = 4), "Order Test")